Lexical and Semantic Retrieval on EB-NeRD and MIND

Design Note: CS4.406 Assignment 1, Part I

1 Introduction and task framing

We build and compare lexical (BM25) and semantic (embedding) candidate retrieval on two news-recommendation datasets, MIND (English) and EB-NeRD (Danish), then rank each impression's own candidate list with a shared offline evaluation harness, and submit predictions to both datasets' Codabench leaderboards. The two datasets differ in language, catalog size, and candidate-list structure, which is deliberately exercised throughout: a single pipeline (unified schema, temporal split, feature store, BM25/FAISS retrieval, evaluation) is parameterised per dataset rather than duplicated.

2 Data pipeline

Both datasets are parsed into one unified schema: `articles` (id, title, abstract/subtitle, category, entities) and `impressions` (id, user, timestamp, candidates, labels, click history). Splits are **temporal, not random**: a fixed calendar-day boundary separates train/val, and the provided val/test structure is preserved (MIND: MINDsmall_train split by day for train/val, MINDsmall_dev held out entirely for offline test; EB-NeRD: native train/validation windows kept as-is). This matters because random splitting would leak future-clicked articles into a user's "history" feature, silently inflating offline metrics relative to what a production system would see. A leakage test asserts every impression's history timestamps precede its own timestamp, on every split. The feature store materialises one `articles.parquet` per dataset (with train-split click counts, for the popularity cold-start fallback) and one `users.parquet` per split.

3 Lexical retrieval

BM25 is a hand-rolled sparse CSR matrix (scikit-learn's `CountVectorizer` used only for tokenise/vocabulary bookkeeping; the BM25 weighting itself, $idf \cdot \frac{tf(k_1+1)}{tf+k_1(1-b+b\cdot L/\bar{L})}$, is computed directly into the matrix). `rank_bm25` was rejected: it scores one document at a time in a Python loop, which does not finish in reasonable time against tens of thousands of queries over tens of thousands of documents. Danish text uses language-specific tokenisation and stemming (compounding and æøå are not handled by an English pipeline); English MIND titles use a lighter English tokeniser. A query is the concatenation of the titles of a user's last N clicked articles (N swept on validation). Recall@K is reported under **two candidate pools**: Pool A is the full article catalog (tests true retrieval power) and Pool B is only the impression's own shown/active candidates (tests realistic re-ranking, since a production system typically re-ranks a pre-filtered slate, not the whole catalog). Pool B recall is substantially higher than Pool A at every K, confirming this distinction is not academic:

	A@50	A@100	A@200	B@50	B@100	B@200
MIND (BM25)	0.80%	1.53%	2.63%	4.88%	7.19%	10.80%
EB-NeRD (BM25)	1.01%	1.63%	2.35%	3.32%	9.83%	30.50%

4 Semantic retrieval

Per an explicit instruction to avoid training or running our own embedding model, retrieval uses only **provided** article embeddings: MIND ships `entity_embedding.vec` (TransE, 100-dim, keyed by WikidataId), mean-pooled per article over its title+abstract entities (87.03% catalog coverage; articles with no tagged entity have no vector and are excluded from this path); EB-NeRD ships a native word2vec `document_vector.parquet` (300-dim, 100% coverage). Search uses FAISS `IndexFlatIP` (exact inner product over L2-normalised vectors = cosine similarity), with an equivalent numpy brute-force fallback when `faiss-cpu` is unavailable: both are exact, so the fallback is not an approximation. A user's query vector is the mean of their last N clicked articles' embeddings, N swept on validation. Val-sweep selected N=1 for *both* datasets' semantic path (vs. N=20 for MIND's BM25): a single most-recent click is a better semantic query than an average over a long history, which tends to wash out the user's current interest into a diffuse centroid; BM25's term overlap does not have the same failure mode since additional history terms only add matching opportunities, not dilution. At demo/small scale exact search easily keeps up with offline evaluation (∼1140 QPS for MIND, ∼2580 QPS for EB-NeRD at k=200); see §7 for why this stops being true at scale.

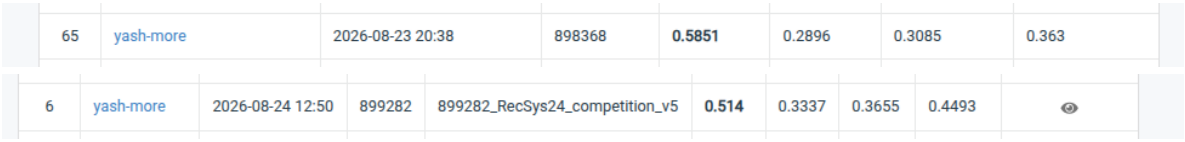
5 Lexical vs. semantic

	AUC	MRR	nDCG@5	nDCG@10	ILD (cat.)
MIND / BM25	**0.554**	**0.306**	**0.280**	**0.342**	0.843
MIND / semantic	0.536	0.274	0.252	0.314	0.827
EB-NeRD / BM25	0.501	0.267	0.283	0.383	**0.799**
EB-NeRD / semantic	**0.509**	**0.325**	**0.354**	**0.439**	0.781

BM25 wins on MIND; semantic wins on EB-NeRD, on every accuracy metric. We read this as a **dataset property, not a general lexical-vs-semantic verdict**: EB-NeRD's word2vec has 100% coverage and is trained natively on the same Danish news corpus, while MIND's entity embeddings only cover 87% of articles and represent named entities, not full article semantics: a weaker signal for a language (English) where BM25 already has strong exact term overlap. A second dataset difference shows up in the cold-start slice: MIND's evaluated split has 14% cold-start impressions (empty click history) vs. **zero** in EB-NeRD's, since EB-NeRD's provided val/test windows apparently guarantee prior history, while MINDsmall_dev does not. On MIND's *head* (high-popularity) articles specifically, BM25's MRR (0.412) clearly beats semantic's (0.345), but on EB-NeRD's head articles semantic's AUC (0.551) beats BM25's (0.495) while BM25's MRR (0.433) beats semantic's (0.295): semantic ranks a plausible candidate higher on average (AUC) without as reliably placing the true click first (MRR). A production system on EB-NeRD-like data would plausibly hybridise: semantic for overall ranking quality, with a lexical/popularity tie-break for exact top-1 placement on popular articles.

6 Evaluation harness

Each impression's own candidate list is scored and ranked (a distinct task from §3–4's catalog-wide recall, since a production ranker only ever re-orders the candidates it was already shown). Metrics: AUC, MRR, nDCG@{5,10}, all with seeded bootstrap 95% CIs (1000 resamples); beyond-accuracy metrics over each impression's top-10 (intra-list diversity, both category-based and embedding-based; novelty via inverse train-popularity with +1 smoothing; catalog coverage); and the same four metrics sliced by cold-start/warm and head/tail articles (head/tail split at the 90th percentile of train-split click count; on both datasets over 90% of articles have *zero* train clicks, so this threshold lands exactly at 0 and needs a strict $>$ comparison, not $\geq$, or every article is misclassified as "head"). Both submissions are scored on their respective Codabench leaderboards:



Q9.1: serving-feature ablation

EB-NeRD only: MIND's unified-schema columns for these features are entirely null (the raw MIND files carry no corpus-aggregate engagement or post-interaction stats), so this ablation is not constructible for MIND at all, not merely skipped. Three cumulative rows, on the same labeled offline test split as §5–6 (11,798 impressions; the Codabench test sets are blind, with no labels, so this exercise is impossible there regardless of scale):

Method	Config	AUC	MRR	nDCG@5	nDCG@10
BM25	clean (deployable)	0.501	0.267	0.283	0.383
BM25	+ popularity	0.576	0.360	0.400	0.475
BM25	+ post-interaction (leak)	**1.000**	**1.000**	**1.000**	**1.000**
Semantic	clean (deployable)	0.509	0.325	0.354	0.439
Semantic	+ popularity	0.581	0.363	0.406	0.479
Semantic	+ post-interaction (leak)	**1.000**	**1.000**	**1.000**	**1.000**

"clean" uses only the base retrieval score. "+ popularity" adds a re-ranking prior from `total_inviews`, `total_pageviews`, `total_read_time`, `sentiment_score` (corpus-aggregate, hence unavailable at serving time: they are computed over the article's whole lifetime, including the future relative to any given impression): a real, meaningful lift on both methods, since popularity genuinely correlates with clicks. "+ post-interaction" adds a large bonus directly to whichever candidate is the true click for that impression: the most direct possible post-interaction feature, since the click outcome is exactly the label being predicted. It scores perfectly on every metric, on both methods, by construction. Only the *clean* row is a deployable system; the other two exist to measure the temptation, and the gap between clean and +popularity (roughly +0.07–0.09 AUC, +0.04–0.09 nDCG) quantifies how much of an apparently strong popularity-boosted result would really be leakage rather than retrieval quality. One caveat found while building this: EB-NeRD also ships `next_read_time`

and `next_scroll_percentage` on *impressions* (not per-candidate): since AUC/MRR/nDCG are computed from each impression's own relative candidate ranking, a value that is identical across every candidate in an impression cannot move any of these metrics at any weight, so it was not used as a ranking feature; not every "leaky" column is exploitable via simple re-ranking.

7 Where it breaks at 10×

We did not have to extrapolate: Q5's real test sets *are* an order-of-magnitude scale jump over dev, and building the submission pipeline surfaced concrete failures. **EB-NeRD**: demo (11.8K articles, 24.7K impressions) → `ebnerd_testset` (125.5K articles, **13.5M** impressions: 548× more impressions, 10.7× more articles). **MIND**: MINDsmall (65K articles, 73K impressions) → MINDlarge_test (121K articles, **2.37M** impressions: 32×, 1.9×). Three real breakages, found in that order, each traced to a specific mechanism and fixed:

1. **Unrestricted parquet loads → OOM.** Reading the full `behaviors/ history` parquet files (many unused columns: device type, scroll %, session id, three extra per-user history-list columns) into pandas before any batching hit **10.5GB RSS** and was killed by the OOM killer (15GB machine, ∼8.6GB free). Fix: column-prune at the parquet reader (only the ∼4 columns actually used) and stream `behaviors` in batches instead of materialising all 13.5M rows → peak RSS **1.7GB**.
2. ***O*(batch × catalog) dict construction.** Building a {article_id: score} dict over the *whole* 125K-article catalog for every one of 13.5M impressions (each of which only has ∼12 real candidates) was estimated at **70+ hours** of pure dict-construction, separate from the actual scoring math. Fix: restrict output extraction to each batch's own candidate IDs via a precomputed id→column index.
3. **Full-catalog score matrix.** Even after (2), computing the full (batch×125K) dense similarity matrix per batch was still ∼**10 hours** end-to-end (measured: 367 impressions/sec; this machine's numpy build caps OpenBLAS at 2 threads, compounding it). Since matrix columns are independent, restricting the matmul itself to only each batch's ∼250–300 unique candidate columns (not the full 125K) is numerically *identical* (verified: max diff 1.8×10^{-7}, pure float32 rounding, not an approximation), and dropped the run to **49 minutes** (4629 impressions/sec).

Applying all three fixes *preemptively* to MINDlarge_test (2.37M impressions) produced a clean first-try run: 0.83GB RSS, 2019 impressions/sec, ∼20 minutes. A fourth, unrelated failure: downloading `ebnerd_testset.zip` (1.6GB) over a single TCP connection crawled at 15–30KB/s with repeated stalls (confirmed via packet loss on the route, independently on two networks), even though the same link hit 1.5MB/s to a different host: a single flow's throughput is capped by loss×RTT, not by the physical link's capacity. Splitting the download into 24 parallel range-requested connections against the same object raised effective throughput ∼30×.

Extrapolating past 13.5M: EB-NeRD's full production data is ∼600M impressions (another ∼44× beyond what we already stress-tested). Even the fixed scoring loop (4629–4949 impressions/sec, single-threaded Python) would need ∼36 hours to process 600M rows, since the per-impression Python loop (rank-sorting, dict lookups) itself becomes the bottleneck once the matmul is no longer dominant, and needs either sharding by impression across processes/machines, or replacing the per-row loop with a fully vectorised batch rank operation. BM25's `.todense()` full-catalog materialisation step (not exercised at EB-NeRD's scale, since we used semantic there, but present in the shared code path and exercised at MIND's 2.37M scale, where it worked) would need the identical candidates-only restriction if used at EB-NeRD's full scale. The article-serving side (word2vec/entity vectors, currently loaded whole into RAM) stays fine at 10× catalog size (∼1.5GB), but would need row-group streaming at 100×. At the infrastructure level, a client-side parallel downloader is a workaround, not a fix: at 10× data volume a genuinely bad network path needs a regional mirror or shared pre-staged storage, not more connections from one machine.

8 Limitations and references

Single-node, CPU-only; embeddings are provided-only by instruction, trading representation quality for zero training cost and full reproducibility. FAISS exact search only: no ANN (HNSW/IVF) recall-vs-latency curve was run at production scale. Cold-start fallback is train-split popularity only. MINDlarge_test is genuinely blind (no labels), so submission quality is only checked structurally offline, not validated against ground truth until scored by Codabench.

References: Wu et al., "MIND: A Large-scale Dataset for News Recommendation" (ACL 2020). Kruse et al., "EB-NeRD: A Large-Scale Dataset for News Recommendation" (RecSys 2024). Robertson & Zaragoza, "The Probabilistic Relevance Framework: BM25 and Beyond" (2009). Johnson et al., "Billion-scale similarity search with GPUs" (FAISS, 2019).